

**PROGRESSIVE FEATURE ENGINEERING AND IMBALANCED
MACHINE LEARNING FOR CREDIT CARD FRAUD
DETECTION**

BY

JOSHUA IBITOYE

MATRIC NO: 225791

A Dissertation in the Department of Computer Science

Submitted to the Faculty of Engineering

in partial fulfilment of the requirements for the Degree of

MASTER OF SCIENCE

in

INFORMATION SECURITY

of the

UNIVERSITY OF IBADAN

[June, 2026]

CERTIFICATION

I certify that this work was carried out by Joshua Ibitoye (Matric No: 225791) in the Department of [Department Placeholder], Faculty of [Faculty Placeholder], University of Ibadan, under my supervision.

Supervisor

[Supervisor Name]

[Supervisor Rank/Title], Department of [Department Placeholder],
University of Ibadan, Ibadan, Nigeria.

Date: _____

DEDICATION

This dissertation is dedicated to the Almighty God, the source of all wisdom and understanding, and to my family, whose unfailing encouragement and sacrifice made this programme of study possible.

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude to my supervisor, [Supervisor Name], for the patient guidance, scholarly direction and constructive criticism that shaped this study from its conception to its conclusion. I am equally grateful to the academic and technical staff of the Department of computer science, Faculty of Engineering, University of Ibadan, for the rich learning environment and the standards of rigour upheld throughout my postgraduate programme.

I appreciate my colleagues in the Master of Science programme in Information Security for the many stimulating discussions that sharpened my understanding of machine learning and information security. Finally, I thank my family and friends for their moral and emotional support during the period of this research. Whatever merit this work possesses is shared with all who supported me; its shortcomings remain entirely my own.

ABSTRACT

Credit card fraud is a costly and continually adapting threat to digital payment systems, and the transaction records used to detect it are severely imbalanced because genuine transactions vastly outnumber fraudulent ones. Under this imbalance, overall accuracy is misleading, and static rule-based controls struggle to track evolving fraud behaviour. The aim of this study was to design, implement and evaluate a reproducible, imbalance-aware machine learning framework for credit card fraud detection that examines how progressive feature engineering and the handling of class imbalance affect fraud-class performance.

The study adopted an experimental quantitative design using a researcher-generated synthetic credit card transaction dataset comprising 499,985 transactions, of which 62,500 (12.5004 per cent) were fraudulent and 437,485 were legitimate, with the binary target FraudFlag. After cleaning, the dataset contained no missing values, no duplicate rows and no duplicate transaction identifiers. Five progressively enriched feature versions (v1 to v5) were constructed from a single clean population, and five classifiers, namely Logistic Regression, Decision Tree, Random Forest, XGBoost and CatBoost, were trained using a stratified 60/20/20 train, validation and test partition. Hyperparameters were tuned by five-fold cross-validation optimising the area under the precision-recall curve, model families were selected on the validation split, and final results were reported once on the untouched test split.

Each configuration was evaluated without and with the Synthetic Minority Over-sampling Technique (SMOTE), applied only within cross-validation folds, and operating thresholds were subsequently tuned on validation predictions. At the default threshold, fraud-class F1 improved as features were enriched, rising from approximately 0.78 for v1 and v2 to about 0.84 to 0.86 for v3 to v5; SMOTE improved default-threshold fraud-class F1 in every version, chiefly by raising precision while modestly reducing recall. After validation-based threshold tuning, the SMOTE and non-SMOTE configurations were practically equal, with the best tuned fraud-class F1 of about 0.862 obtained by the v5 non-SMOTE Random Forest at a threshold of 0.80 (precision 0.999, recall 0.758).

The principal finding is that careful feature engineering and threshold selection, rather than over-sampling alone, drive fraud-class performance on this dataset, and that the

strongest versions (v3 to v5) are practically tied. The study contributes a transparent, leakage-aware and reproducible evaluation pipeline for imbalanced tabular fraud detection. Its conclusions are bounded by important limitations: the data are synthetic, no external validation set was used, the MerchantRisk and CardRisk indicators carry a residual leakage risk, and no deployment or real-time evaluation was performed.

Keywords: *credit card fraud detection, class imbalance, SMOTE, threshold optimisation, precision-recall AUC.*

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
AUC	Area Under the Curve
CV	Cross-Validation
F1	Harmonic mean of precision and recall (F1-score)
FN	False Negative
FP	False Positive
ML	Machine Learning
PR-AUC	Area Under the Precision-Recall Curve
RF	Random Forest
ROC	Receiver Operating Characteristic
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
TN	True Negative
TP	True Positive
UIMS	University of Ibadan Manual of Style
XGBoost	Extreme Gradient Boosting

TABLE OF CONTENTS

Certification.....	iii
Dedication	iv
Acknowledgements	v
Abstract.....	vi
List of Abbreviations	viii
List of Tables	xii
List of Figures.....	xiii
CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem	2
1.3 Aim of the Study	3
1.4 Objectives of the Study	3
1.5 Research Questions	3
1.6 Significance of the Study	4
1.7 Scope of the Study.....	4
1.8 Limitations of the Study	4
1.9 Definition of Terms	5
CHAPTER TWO: LITERATURE REVIEW	7
2.1 Conceptual Review.....	7
2.2 Theoretical Review.....	9
2.3 Empirical Review	10
2.4 Gap in Literature	12
2.5 Summary of Literature Review	12
CHAPTER THREE: METHODOLOGY.....	13
3.1 Research Design	13
3.2 Dataset Generation and Description.....	13
3.3 Dataset Cleaning and Validation	14

3.4 Progressive Feature Engineering	14
3.5 Data Preprocessing	15
3.6 Train, Validation and Test Splitting	16
3.7 Model Development	16
3.8 SMOTE Configuration	17
3.9 Hyperparameter Tuning	17
3.10 Threshold Optimisation	17
3.11 Evaluation Metrics	17
3.12 Experimental Workflow	18
3.13 Ethical Considerations	19
CHAPTER FOUR: RESULTS AND DISCUSSION	20
4.1 Dataset Audit Results	20
4.2 Class Distribution	20
4.3 Dataset Version Summary	21
4.4 Default-Threshold Results Without SMOTE	21
4.5 Default-Threshold Results With SMOTE	22
4.6 Effect of Progressive Feature Engineering	22
4.7 Effect of SMOTE	24
4.8 Threshold-Tuned Results	25
4.9 Best Model Interpretation	26
4.10 Discussion of Findings in Relation to Research Questions	27
4.11 Summary of Chapter	28
CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS	29
5.1 Summary of the Study	29
5.2 Summary of Findings	29
5.3 Conclusion	29
5.4 Contributions to Knowledge	30
5.5 Recommendations	30

5.6 Suggestions for Future Work	30
REFERENCES.....	32
APPENDICES.....	35
Appendix A: Dataset Features / Data Dictionary	35
Appendix B: Experimental Configuration	35
Appendix C: Additional Metrics	36
Appendix D: Reproducibility Notes.....	36
Appendix E: Selected Code / Workflow Summary.....	37

LIST OF TABLES

Table 2.1: Empirical Review Comparison Matrix	11
Table 3.1: Dataset Summary	13
Table 3.2: Progressive Dataset Versions.....	15
Table 3.3: Experimental Configuration.....	18
Table 4.1: Dataset Audit Summary	20
Table 4.2: Best Default-Threshold Results Without SMOTE.....	21
Table 4.3: Best Default-Threshold Results With SMOTE.....	22
Table 4.4: Threshold-Tuned Comparison	25
Table 4.5: Summary of Findings by Research Question.....	28

LIST OF FIGURES

Fig. 2.1: Conceptual framework for the study	9
Fig. 3.1: Experimental workflow.....	18
Fig. 3.2: Progressive dataset version design.....	15
Fig. 4.1: Class distribution of the dataset	21
Fig. 4.2: Fraud-class F1 comparison.....	23
Fig. 4.3: Fraud-class precision comparison	23
Fig. 4.4: Fraud-class recall comparison.....	24
Fig. 4.5: PR-AUC comparison	24
Fig. 4.6: Threshold optimisation curve (v5 No-SMOTE Random Forest).....	25
Fig. 4.7: Confusion matrix for the best tuned model	26
Fig. 4.8: Feature importance for the v5 No-SMOTE Random Forest	27

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

The global migration of commerce to digital channels has made the payment card one of the most important instruments of modern economic life. Electronic and card-based payments have grown rapidly as consumers and businesses embrace online retail, mobile commerce and cashless transactions, and this expansion has been accompanied by a corresponding rise in the volume and sophistication of payment card fraud (Bhattacharyya, Jha, Tharakunnel and Westland, 2011; West and Bhattacharya, 2016). Fraudulent use of card credentials imposes direct financial losses on issuers, merchants and cardholders, and it erodes the trust on which digital payment ecosystems depend.

Credit card fraud detection is naturally framed as a binary classification problem in which each transaction is labelled either legitimate or fraudulent (Ngai, Hu, Wong, Chen and Sun, 2011). Early defences relied heavily on static, rule-based systems in which human experts encoded heuristics such as spending thresholds or blacklisted merchants. Although interpretable, such systems are brittle: they require continual manual maintenance, they generalise poorly to previously unseen fraud patterns, and fraudsters adapt their behaviour to evade fixed rules (Abdallah, Maarof and Zainal, 2016). These weaknesses have motivated a shift towards data-driven machine learning, which can learn complex, non-linear relationships from historical transactions and adapt as new labelled examples become available (Phua, Lee, Smith and Gayler, 2010).

A defining characteristic of fraud detection data is extreme class imbalance: fraudulent transactions typically constitute only a very small fraction of all transactions (Chawla, Bowyer, Hall and Kegelmeyer, 2002; He and Garcia, 2009). This imbalance has two important consequences. First, overall classification accuracy becomes a misleading measure of performance, because a trivial model that predicts every transaction as legitimate can attain very high accuracy while detecting no fraud at all (Saito and Rehmsmeier, 2015). Second, learning algorithms trained without imbalance-aware strategies tend to be biased towards the majority legitimate class, depressing recall on the minority fraud class (Fernandez, Garcia, Galar, Prati, Krawczyk and Herrera, 2018). Researchers therefore emphasise minority-sensitive evaluation metrics such as

fraud-class precision, recall and F1-score, together with the area under the precision-recall curve (PR-AUC), which is more informative than the receiver operating characteristic curve when the positive class is rare (Davis and Goadrich, 2006).

Two further levers strongly influence performance on imbalanced tabular data: feature engineering and the handling of imbalance itself. Thoughtfully engineered features that capture transaction context, such as the relationship between a transaction amount and its category norm, can substantially improve the separability of the minority class (Whitrow, Hand, Juszczak, Weston and Adams, 2009; Bahnsen, Aouada, Stojanovic and Ottersten, 2016). Imbalance can be addressed at the data level through resampling techniques such as the Synthetic Minority Over-sampling Technique (SMOTE), or at the algorithm level through class weighting and decision-threshold adjustment (Chawla et al., 2002; Krawczyk, 2016). However, the benefits of these techniques are not universal, and their effect is closely tied to the operating threshold at which the classifier is finally deployed. This study examines these levers systematically and under controlled conditions.

1.2 Statement of the Problem

Credit card fraud remains a costly and continually adapting threat to digital financial systems. The datasets used to study it are intrinsically imbalanced, and this imbalance makes accuracy an unreliable indicator of true detection capability. Static, rule-based controls cannot keep pace with evolving fraud strategies, and machine learning approaches, while promising, are frequently reported in ways that limit their scientific value.

Three specific gaps motivate this work. First, many studies report results without a rigorous, leakage-aware experimental protocol, so it is difficult to know whether reported gains reflect genuine learning or information leakage from improper preprocessing or evaluation (Kaufman, Rosset, Perlich and Stitelman, 2012). Second, the value of resampling techniques such as SMOTE is often asserted at a single default threshold, even though the apparent benefit may disappear once the decision threshold is tuned for the operating point of interest. Third, the contribution of progressive feature engineering is rarely isolated from confounding factors such as changes in dataset size or composition. There is therefore a need for a transparent and reproducible tabular machine learning pipeline that holds the data population constant,

varies feature richness in a controlled manner, compares imbalance strategies fairly, and reports minority-class performance honestly both at the default threshold and after threshold optimisation.

1.3 Aim of the Study

The aim of this study is to design, implement, and evaluate a machine learning-based framework for credit card fraud detection using progressive feature engineering and imbalance-aware evaluation.

1.4 Objectives of the Study

The specific objectives of the study are to:

1. generate and clean a synthetic credit card transaction dataset;
2. construct progressively enriched dataset versions;
3. preprocess numeric and categorical transaction features;
4. implement Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost models;
5. evaluate model performance under no-SMOTE and SMOTE settings;
6. perform validation-based threshold optimisation;
7. compare models using fraud-class precision, recall, F1-score, ROC-AUC, and PR-AUC; and
8. identify the most effective model-version configuration under controlled experimental conditions.

1.5 Research Questions

In line with the objectives, the study addresses the following research questions:

- RQ1. Can a synthetic credit card transaction dataset be generated, cleaned and validated to provide a sound and reproducible basis for fraud-detection experiments?
- RQ2. How does progressive feature engineering across dataset versions v1 to v5 affect fraud-class detection performance?
- RQ3. How do Logistic Regression, Decision Tree, Random Forest, XGBoost and CatBoost compare on the fraud-detection task under controlled conditions?

RQ4. What is the effect of SMOTE on fraud-class performance at the default decision threshold?

RQ5. Does validation-based threshold optimisation alter the comparison between the SMOTE and non-SMOTE configurations?

RQ6. Which model-version configuration provides the most effective fraud-class performance under the experimental conditions of this study?

1.6 Significance of the Study

The study is significant in several respects. Academically, it contributes a controlled experimental treatment of progressive feature engineering and imbalance handling, isolating their effects by holding the underlying data population constant across feature versions. Methodologically, it demonstrates a leakage-aware pipeline in which distribution-dependent features are recomputed within cross-validation folds, model families are selected on a dedicated validation split, and final performance is reported once on an untouched test split; this design offers a template for reproducible fraud-detection research. Practically, the comparative evidence on SMOTE and threshold tuning informs how practitioners might configure imbalance-aware classifiers and select operating points. Societally, more dependable fraud detection supports the integrity of secure digital financial systems, on which individuals, businesses and the wider economy increasingly rely.

1.7 Scope of the Study

The study is limited to supervised, tabular machine learning for binary fraud classification. It is an offline experimental study conducted on a researcher-generated synthetic credit card transaction dataset. It does not involve integration with any live banking system, and it does not perform real-time deployment or streaming evaluation. The study does not involve natural language processing or any form of unstructured-text analysis; all features are structured transaction attributes. The experimental scope is confined to five classifiers and five progressive feature versions, evaluated with and without SMOTE and with validation-based threshold optimisation.

1.8 Limitations of the Study

The principal limitations of the study are acknowledged at the outset. The dataset is synthetic and researcher-generated, so its statistical structure may not fully reflect the

complexity of real institutional transaction streams. No external, independently sourced validation dataset was available, so generalisation beyond the present data cannot be demonstrated. No live or real-time deployment was undertaken. Two engineered indicators, MerchantRisk and CardRisk, are proxy risk features that carry a residual risk of information leakage and are therefore interpreted with caution. Repeated-seed robustness analysis and formal confidence intervals were not computed within the present pipeline, and a full regulatory or legal analysis of deployment was outside the study's remit. Finally, the study does not implement deep learning or graph-based fraud models.

1.9 Definition of Terms

Credit card fraud: the unauthorised use of credit card credentials to obtain goods, services or funds, treated here as the positive class in a binary classification task.

Fraud-class F1: the harmonic mean of precision and recall computed specifically for the fraudulent (minority) class; the primary performance criterion in this study.

Class imbalance: a condition in which the classes are very unevenly represented, here with fraudulent transactions forming a small minority of all records.

SMOTE: the Synthetic Minority Over-sampling Technique, which generates synthetic minority-class examples by interpolating between existing minority instances and their neighbours.

PR-AUC: the area under the precision-recall curve, an imbalance-sensitive summary of the trade-off between precision and recall across thresholds.

ROC-AUC: the area under the receiver operating characteristic curve, summarising the trade-off between true-positive and false-positive rates across thresholds.

Precision: the proportion of transactions predicted as fraudulent that are truly fraudulent.

Recall: the proportion of truly fraudulent transactions that are correctly identified.

Threshold optimisation: the selection of the probability cut-off used to convert predicted probabilities into class labels, here chosen on validation data and evaluated on test data.

Feature engineering: the construction of informative input variables from raw transaction attributes to improve a model's ability to separate the classes.

Random Forest: an ensemble of decision trees trained on bootstrapped samples with random feature selection, aggregated to reduce variance.

XGBoost: a scalable and regularised implementation of gradient-boosted decision trees.

CatBoost: a gradient-boosting algorithm with native and ordered handling of categorical features designed to reduce prediction-shift bias.

CHAPTER TWO

LITERATURE REVIEW

This chapter reviews the conceptual foundations, theoretical underpinnings and empirical evidence relevant to machine learning-based credit card fraud detection. It then identifies the gap in the literature that the present study addresses and summarises the review.

2.1 Conceptual Review

2.1.1 Credit Card Fraud

Credit card fraud refers to the unauthorised use of card credentials to obtain goods, services or funds. It encompasses card-present and card-not-present schemes, the latter having grown with online commerce. From an analytical perspective, the task of detecting such fraud is a binary classification problem in which the rare positive class, fraud, must be distinguished from a dominant negative class of legitimate activity (Ngai et al., 2011; Abdallah et al., 2016). Because fraud behaviour evolves to evade controls, detection models must capture subtle and shifting patterns rather than fixed signatures.

2.1.2 Fraud Detection Systems

Fraud detection systems have historically combined rule-based engines with manual review. Rule-based systems encode expert heuristics and are transparent, but they are costly to maintain and generalise poorly to novel fraud (Phua et al., 2010). Data-driven systems, by contrast, learn decision boundaries from labelled history and can be retrained as new evidence accumulates. Contemporary architectures increasingly blend supervised classifiers with unsupervised anomaly detection to capture both known and emerging fraud (Carcillo, Le Borgne, Caelen, Kessaci, Oble and Bontempi, 2021).

2.1.3 Machine Learning in Fraud Detection

Machine learning methods learn statistical relationships between transaction attributes and the fraud label. Linear models such as logistic regression provide interpretable baselines (Cox, 1958), while tree-based methods and ensembles capture non-linear interactions that are common in tabular financial data (Breiman, 2001; Chen and Guestrin, 2016). Reviews of the field report that ensemble and boosting methods

frequently achieve the strongest tabular performance, although results depend heavily on data quality, feature design and evaluation protocol (West and Bhattacharya, 2016).

2.1.4 Class Imbalance in Fraud Detection

The defining statistical challenge in fraud detection is severe class imbalance (Chawla et al., 2002; He and Garcia, 2009). Models trained naively on imbalanced data tend to favour the majority class and under-detect fraud. Remedies operate at the data level, for example over-sampling the minority class with SMOTE or under-sampling the majority class, and at the algorithm level, for example through class weighting and threshold adjustment (Krawczyk, 2016; Fernandez et al., 2018; Branco, Torgo and Ribeiro, 2016). The literature cautions that no single remedy is universally superior and that the chosen evaluation protocol strongly conditions the apparent benefit of any technique.

2.1.5 Feature Engineering for Tabular Fraud Detection

Feature engineering is widely regarded as decisive for tabular fraud detection. Aggregation features that summarise a cardholder's recent behaviour, and contextual features that compare a transaction with category or peer norms, can markedly improve minority-class separability (Whitrow et al., 2009; Bahnsen, Aouada, Stojanovic and Ottersten, 2016). Engineered ratios, such as the relationship between a transaction amount and the typical amount for its category, frequently rank among the most informative predictors. The present study operationalises this insight through a progression of feature versions of increasing richness.

2.1.6 Threshold Optimisation

Most probabilistic classifiers output a fraud probability that must be converted into a decision by comparing it with a threshold. The conventional default of 0.5 is rarely optimal under imbalance, and selecting the threshold for the desired operating point can change measured performance considerably (Hand, 2009; Provost and Fawcett, 2001). Sound practice selects the threshold on data not used for final reporting, so that the chosen operating point is evaluated honestly. This study selects thresholds on a validation split and reports them on an untouched test split.

Figure 2.1 presents the conceptual framework adopted for the study. It links transaction evidence to engineered features, the handling of class imbalance, the family of learning algorithms, and the fraud-class performance that constitutes the study's

outcome of interest. The framework expresses the central hypothesis that feature richness and operating -point selection, rather than resampling alone, govern minority-class performance.

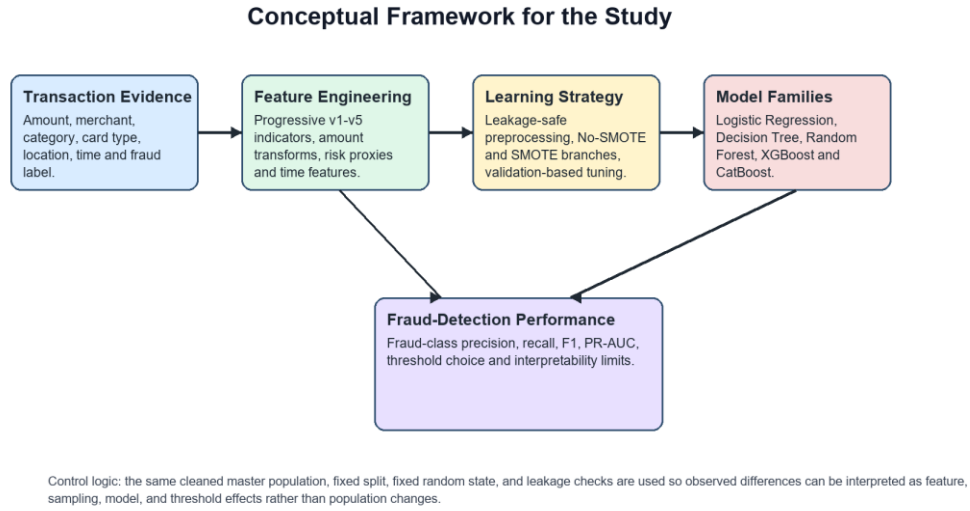


Fig. 2.1: Conceptual framework linking transaction evidence, feature engineering, imbalance handling, model learning and fraud-class performance.

2.2 Theoretical Review

2.2.1 Supervised Learning Theory

Supervised learning seeks a function that maps inputs to outputs by minimising expected loss, estimated empirically from labelled training data, while controlling model complexity to avoid overfitting (Hastie, Tibshirani and Friedman, 2009; James, Witten, Hastie and Tibshirani, 2013). The bias-variance trade-off provides the conceptual basis for regularisation, cross-validation and ensembling, all of which are employed in this study.

2.2.2 Statistical Decision Theory

Statistical decision theory frames classification as choosing actions to minimise expected cost given posterior class probabilities. Under imbalance and unequal misclassification costs, the optimal decision threshold departs from 0.5, which provides the theoretical justification for threshold optimisation and for evaluating precision and recall rather than accuracy alone (Hand, 2009; Provost and Fawcett, 2001).

2.2.3 Ensemble Learning Theory

Ensemble theory explains why combining many weak learners reduces generalisation error. Bagging, exemplified by the Random Forest, reduces variance by averaging decorrelated trees (Breiman, 2001), whereas boosting, exemplified by gradient boosting and its scalable implementations, reduces bias by sequentially fitting residuals (Friedman, 2001; Chen and Guestrin, 2016; Prokhorenkova, Gusev, Vorobev, Dorogush and Gulin, 2018). These complementary mechanisms motivate the inclusion of both bagged and boosted models.

2.2.4 Imbalanced Learning Theory

Imbalanced learning theory characterises how skewed priors distort decision boundaries and degrade minority-class recall, and it formalises data-level and algorithm-level corrections (He and Garcia, 2009; Fernandez et al., 2018). It also clarifies why threshold-independent, imbalance-sensitive metrics such as PR-AUC give a more faithful picture of minority-class performance than accuracy or ROC-AUC when the positive class is rare (Davis and Goadrich, 2006; Saito and Rehmsmeier, 2015).

2.3 Empirical Review

A substantial body of empirical research has examined machine learning for fraud detection, imbalance handling and evaluation. Bhattacharyya et al. (2011) compared logistic regression, decision trees and support vector machines on real card data and reported the strength of data-mining approaches, while highlighting reproducibility constraints arising from data confidentiality. Whitrow et al. (2009) demonstrated that transaction aggregation features improve detection, underscoring the centrality of feature design. Dal Pozzolo, Caelen, Johnson and Bontempi (2015) showed that calibrating probabilities under undersampling improves ranking quality, and Dal Pozzolo, Boracchi, Caelen, Alippi and Bontempi (2018) argued for realistic modelling that accounts for delayed labels and concept drift. Carcillo et al. (2021) combined unsupervised and supervised signals to improve detection. Jurgovsky et al. (2018) treated detection as sequence classification using recurrent networks, illustrating the value of temporal context.

On the algorithmic side, Breiman (2001) introduced the Random Forest, Chen and Guestrin (2016) presented XGBoost as a scalable and regularised boosting system, and

Prokhorenkova et al. (2018) proposed CatBoost with ordered boosting to handle categorical variables while mitigating prediction-shift bias. On evaluation, Davis and Goadrich (2006) and Saito and Rehmsmeier (2015) established that precision-recall analysis is more informative than ROC analysis under heavy imbalance. Bahnsen et al. (2016) advanced example-dependent cost-sensitive learning, and broad surveys by Fernandez et al. (2018) and West and Bhattacharya (2016) synthesised the methodological landscape, concluding that method effectiveness is highly context-dependent. Table 2.1 summarises representative studies across these themes.

Table 2.1: Empirical Review Comparison Matrix

Study	Method / Focus	Evidence Base	Key Finding	Limitation
Chawla et al. (2002)	SMOTE over-sampling	Benchmark imbalanced data	Synthetic minority samples raise recall	May blur class boundaries
Whitrow et al. (2009)	Transaction aggregation features	Real card transactions	Aggregation improves detection	Strongly design-dependent
Bhattacharyya et al. (2011)	LR, DT, SVM comparison	Real card transactions	Data-mining methods effective	Limited public reproducibility
Dal Pozzolo et al. (2015)	Undersampling + calibration	Real European card data	Calibration aids ranking	Undersampling discards data
Davis and Goadrich (2006)	PR vs ROC analysis	Theory and benchmarks	PR more informative under imbalance	Threshold dependence remains
Saito and Rehmsmeier (2015)	PR vs ROC evaluation	Imbalanced benchmarks	PR plot preferred under skew	Requires careful interpretation
Breiman (2001)	Random Forest (bagging)	General benchmarks	Variance reduction, robustness	Reduced interpretability
Chen and Guestrin (2016)	XGBoost (boosting)	General benchmarks	Scalable, strong on tabular data	Sensitive to tuning
Prokhorenkova et al. (2018)	CatBoost (ordered boosting)	Categorical-rich data	Reduces categorical leakage bias	Higher compute cost
Bahnsen et al. (2016)	Cost-sensitive features	Real card transactions	Improves cost-based savings	Needs reliable cost data
Dal Pozzolo et al. (2018)	Realistic learning strategy	Real card transactions	Models drift and delayed labels	Operationally complex
Jurgovsky et al. (2018)	LSTM sequence model	Real card transactions	Temporal context aids detection	Requires sequential data
Carcillo et al. (2021)	Hybrid unsupervised + supervised	Real card transactions	Combined signals improve recall	Increased system complexity
Fernandez et al. (2018)	Imbalanced-learning survey	Broad literature	Taxonomy of remedies	No universally best method
West and Bhattacharya (2016)	Fraud-detection review	Broad literature	ML broadly effective	Data scarcity and privacy

2.4 Gap in Literature

Although the literature is rich, several gaps persist. First, few studies isolate the effect of progressive feature engineering by holding the underlying data population constant while varying only feature richness; consequently, reported gains may be confounded by changes in data size or composition. Second, the value of SMOTE is frequently judged at a single default threshold, even though its apparent advantage may not survive validation-based threshold tuning. Third, reproducible dataset auditing, including explicit removal of duplicate records and leakage-aware feature recomputation within cross-validation folds, is inconsistently reported (Kaufman et al., 2012). Fourth, evaluation continues to rely on accuracy or ROC-AUC in settings where fraud-class metrics and PR-AUC would be more appropriate. This study addresses these gaps through a controlled, leakage-aware and fully documented experimental pipeline.

2.5 Summary of Literature Review

The reviewed literature establishes credit card fraud detection as an imbalanced binary classification problem in which feature engineering, imbalance handling, model choice and evaluation protocol jointly determine performance. Tree-based ensembles and boosting methods are consistently competitive on tabular data, precision-recall analysis is preferred under imbalance, and the benefit of resampling is contingent on the operating threshold. These insights motivate the controlled experimental design, the leakage-aware pipeline and the fraud-class, threshold-aware evaluation adopted in the chapters that follow.

CHAPTER THREE

METHODOLOGY

This chapter describes the research design and the methods used to generate and validate the dataset, engineer progressive feature versions, preprocess the data, train and tune the classifiers, handle class imbalance, optimise decision thresholds, and evaluate model performance. The procedures were implemented as a documented and reproducible pipeline.

3.1 Research Design

The study adopts an experimental, quantitative research design. The experiment manipulates two factors, namely feature richness across five dataset versions and the imbalance-handling strategy (without and with SMOTE), and measures their effect on fraud-class performance under controlled conditions. All feature versions are derived from a single clean data population so that observed differences are attributable to feature engineering rather than to changes in data size or composition. The design is fully computational and offline; it involves no human participants and no live financial systems.

3.2 Dataset Generation and Description

The study uses a researcher-generated synthetic credit card transaction dataset. The dataset was produced programmatically to emulate the structure of card transaction records while containing no personal or institutional data. The final dataset comprises 499,985 transactions, of which 62,500 (12.5004 per cent) are fraudulent and 437,485 are legitimate. The binary target variable is FraudFlag, where 1 denotes a fraudulent transaction and 0 a legitimate one. The dataset and its core characteristics are summarised in Table 3.1.

Table 3.1: Dataset Summary

Attribute	Value
Total transactions	499,985
Fraudulent transactions	62,500
Legitimate transactions	437,485
Fraud ratio	12.5004%
Target label	FraudFlag (1 = fraud, 0 = legitimate)

Attribute	Value
Duplicate rows (final)	0
Duplicate TransactionID (final)	0
Missing values	0
Task type	Binary tabular classification
Dataset type	Synthetic / researcher-generated

3.3 Dataset Cleaning and Validation

Before modelling, the data underwent cleaning and validation to ensure integrity. A clean master population was formed and audited. The provenance source from which the master was consolidated contained 849,999 rows, of which 350,014 were exact duplicate records; these duplicates, together with duplicate transaction identities, were removed so that the final modelling population contained unique records only. The stacked provenance file is retained solely for audit and is not treated as an additional experiment. Each transaction carries a deterministic TransactionID derived by hashing stable transaction fields, which supports duplicate detection and verification that the training, validation and test partitions do not share records. Validation confirmed that the final datasets contain no missing values, no duplicate rows and no duplicate TransactionID, and that categorical text fields were normalised. Improper preprocessing is a common source of data leakage (Kaufman et al., 2012); the pipeline was therefore designed so that all distribution-dependent computations are performed within training folds, as described in Section 3.5.

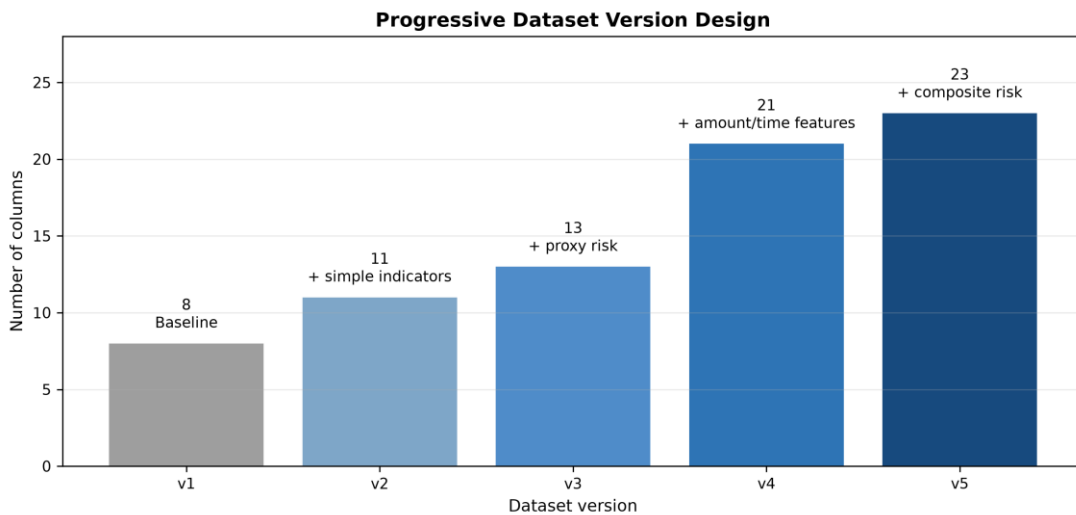
3.4 Progressive Feature Engineering

Five dataset versions of increasing feature richness were constructed from the same clean population. Version v1 contains the baseline transaction, categorical and time fields. Version v2 adds a high-amount indicator, a night-transaction indicator and a transaction subcategory. Version v3 adds merchant and card risk indicators. Version v4 adds amount and time-derived engineered features, including a logarithmic amount transform and amount ratios and standardised scores relative to category norms. Version v5 adds a combined night-and-high-amount indicator and a composite risk score. All versions retain the same 499,985 records and the same fraud ratio, isolating the effect of feature engineering. The progression is detailed in Table 3.2 and illustrated in Figure 3.2.

Table 3.2: Progressive Dataset Versions

Version	Columns	Newly Added Features	Rationale
v1	8	Baseline: Amount, Merchant, Category, CardType, Location, Time	Raw transactional baseline
v2	11	HighAmountFlag, IsNightTransaction, SubCategory	Simple risk and context flags
v3	13	MerchantRisk, CardRisk	Proxy risk indicators (leakage-cautious)
v4	21	Hour, AmountLog, AmountToCategoryMedian, AmountZScoreByCategory, AmountPercentile (and calendar fields, excluded from models)	Amount and time context features
v5	23	NightHighAmountFlag, RiskScoreComposite	Interaction and composite risk features

Figure 3.2 depicts the progressive enrichment of the feature space from v1 to v5 while the underlying records remain constant.

**Fig. 3.2:** Progressive dataset version design from v1 to v5.

3.5 Data Preprocessing

Preprocessing was tailored to each model family and executed inside the modelling pipeline to prevent leakage. For Logistic Regression, categorical variables were imputed with the most frequent value and one-hot encoded, while numeric variables were imputed with the median and standardised. For the tree-based models (Decision Tree, Random Forest and XGBoost), categorical variables were imputed and frequency-encoded, which avoids very high-dimensional sparse matrices, while numeric variables were median-imputed without scaling, since tree splits are scale-invariant. For CatBoost, categorical variables were handled natively by the algorithm's ordered target-based encoding; when SMOTE was applied, CatBoost was trained

within the same imbalanced-learning pipeline using frequency-encoded categoricals. A dedicated, leakage-safe feature-engineering step recomputed all distribution-dependent features, such as category medians, z-scores, percentiles and the composite risk score, using only training-fold statistics, so that no information from validation or test data influenced the engineered values. The columns `FraudFlag`, `TransactionID`, `Time`, `DayOfWeek`, `Month`, `IsWeekend` and `IsWeekendDerived` were excluded from the model feature set; the calendar fields were excluded because they reflected dataset-construction artefacts rather than defensible predictive signals.

3.6 Train, Validation and Test Splitting

The data were partitioned using a stratified 60/20/20 scheme that preserved the fraud ratio in every partition. This produced 299,991 training records, 99,997 validation records and 99,997 test records. The training split was used for model fitting and cross-validated hyperparameter tuning, the validation split was used for model-family selection and threshold selection, and the test split was held out and used once for final reporting. Because each record carries a unique `TransactionID`, the absence of overlap between the partitions was verified.

3.7 Model Development

Five classifiers were implemented. Logistic Regression served as an interpretable linear baseline with balanced class weighting (Cox, 1958). The Decision Tree provided a non-linear, interpretable model with balanced class weighting (Quinlan, 1986). The Random Forest aggregated many decorrelated trees to reduce variance (Breiman, 2001). XGBoost implemented regularised gradient boosting with scalable training (Chen and Guestrin, 2016). CatBoost implemented ordered gradient boosting with native categorical handling (Prokhorenkova et al., 2018). Together these span linear, single-tree, bagged and boosted model families, enabling a fair comparison across paradigms.

3.8 SMOTE Configuration

Class imbalance was addressed using the Synthetic Minority Over-sampling Technique (Chawla et al., 2002), implemented through the `imbalanced-learn` library (Lemaitre, Nogueira and Aridas, 2017). To prevent leakage, SMOTE was placed inside the modelling pipeline and applied only after preprocessing and only within cross-

validation training folds; it was never applied before splitting and never to validation or test data. Each experiment was run twice, once without SMOTE (using class weighting where applicable) and once with SMOTE, to permit a controlled comparison of the two imbalance strategies.

3.9 Hyperparameter Tuning

Hyperparameters were tuned using randomised search with five-fold stratified cross-validation on the training split. The optimisation objective was the area under the precision-recall curve (average precision), which is the appropriate criterion under heavy class imbalance because it focuses on minority-class ranking quality rather than on overall accuracy (Davis and Goadrich, 2006; Saito and Rehmsmeier, 2015). Search spaces covered the principal complexity and regularisation parameters of each model family.

3.10 Threshold Optimisation

Because the default 0.5 threshold is rarely optimal under imbalance, the decision threshold was tuned for each best model. Candidate thresholds were evaluated on validation predictions, and the threshold maximising fraud-class F1 was selected. The chosen threshold was then applied once to the test predictions, so that the reported operating point reflects performance on data not used for threshold selection. This protocol provides an honest estimate of deployable performance at a chosen operating point (Hand, 2009).

3.11 Evaluation Metrics

Model performance was assessed with metrics suited to imbalanced classification. Accuracy was recorded for completeness but was not used for model selection, since it is misleading under imbalance. The primary criterion was fraud-class F1, the harmonic mean of fraud-class precision and recall. Precision measures the proportion of predicted frauds that are truly fraudulent, and recall the proportion of true frauds detected. The area under the receiver operating characteristic curve (ROC-AUC) and the area under the precision-recall curve (PR-AUC) summarised threshold-independent performance, with PR-AUC given primacy under imbalance. The confusion matrix reported true positives, true negatives, false positives and false negatives. Model-family selection followed a documented rule that prioritised fraud-class F1, then PR-

AUC, then recall, then precision, then ROC-AUC, then lower training time, with practical tolerances to avoid over-interpreting negligible differences.

3.12 Experimental Workflow

The end-to-end experimental workflow is shown in Figure 3.1. It proceeds from dataset generation and cleaning, through progressive feature versioning, stratified splitting, model tuning by cross-validation, and validation-based model and threshold selection, to final evaluation on the untouched test split and the comparison of the SMOTE and non-SMOTE configurations.

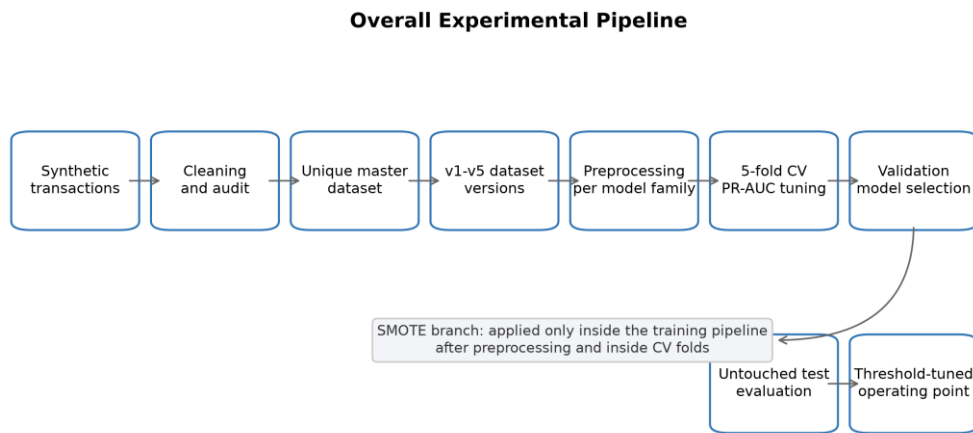


Fig. 3.1: Experimental workflow for dataset preparation, model tuning, evaluation, SMOTE comparison and threshold optimisation.

The principal experimental settings are consolidated in Table 3.3.

Table 3.3: Experimental Configuration

Component	Configuration
Split strategy	Stratified train/validation/test split
Split ratio	60/20/20
Training records	299,991
Validation records	99,997
Test records	99,997
Cross-validation	5-fold CV on training split
Tuning metric	PR-AUC / average precision
Model-family selection	Validation split
Final reporting	Untouched test split
SMOTE placement	Inside imblearn pipeline, after preprocessing, within CV folds
Threshold selection	Maximise fraud-class F1 on validation; report on test

Component	Configuration
Excluded model features	FraudFlag, TransactionID, Time, DayOfWeek, Month, IsWeekend, IsWeekendDerived

3.13 Ethical Considerations

The study raises no human-subjects concerns because it uses a researcher-generated synthetic dataset that contains no personal, financial or institutional data; this design protects privacy by construction. The work is presented as a controlled scientific experiment rather than a deployable system, and no claim of real-world deployment, real-time operation or use of genuine bank data is made. Responsible machine learning practice was observed through leakage-aware preprocessing, honest minority-class reporting and explicit disclosure of limitations. Fairness considerations are acknowledged: although the synthetic data carry no protected attributes, any prospective real-world adaptation would require a fairness and bias audit, a leakage audit of proxy risk indicators, and adherence to applicable data-protection and financial regulations before deployment could be contemplated.

CHAPTER FOUR

RESULTS AND DISCUSSION

This chapter presents and discusses the experimental results. It reports the dataset audit, the class distribution and version design, the default-threshold results without and with SMOTE, the effect of progressive feature engineering, the effect of SMOTE, the threshold-tuned results, and the interpretation of the best model, before relating the findings to the research questions.

4.1 Dataset Audit Results

The dataset audit confirmed the integrity of the modelling data. The provenance source from which the clean master was consolidated contained 849,999 rows, of which 350,014 were exact duplicates; after their removal, together with duplicate transaction identities, the final modelling population contained 499,985 unique records. The audit verified that the final datasets contained no duplicate rows, no duplicate transaction identifiers and no missing values. The audit outcomes are summarised in Table 4.1.

Table 4.1: Dataset Audit Summary

Audit Item	Result
Provenance source rows	849,999
Exact duplicate rows removed	350,014
Final clean records	499,985
Fraudulent transactions	62,500
Legitimate transactions	437,485
Fraud ratio	12.5004%
Duplicate rows (final)	0
Duplicate TransactionID (final)	0
Missing values (final)	0

4.2 Class Distribution

The class distribution reflects the severe imbalance characteristic of fraud-detection data. Fraudulent transactions account for 12.5004 per cent of records, while legitimate transactions account for the remaining 87.4996 per cent. This imbalance, illustrated in Figure 4.1, underlies the study's emphasis on fraud-class metrics and PR-AUC rather than overall accuracy.

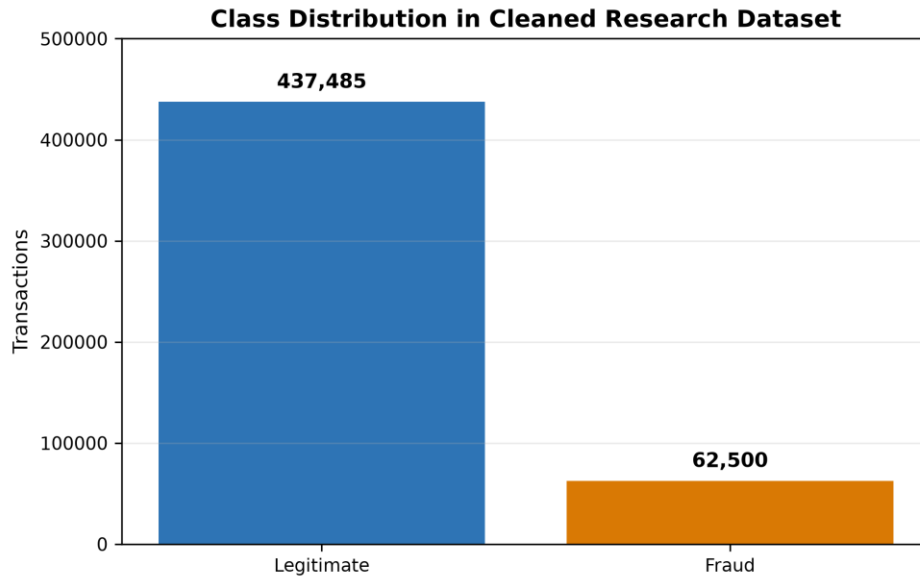


Fig. 4.1: Class distribution of the synthetic credit card transaction dataset.

4.3 Dataset Version Summary

The five feature versions, introduced in Section 3.4 and detailed in Table 3.2 and Figure 3.2, share an identical population of 499,985 records and an identical fraud ratio, differing only in feature richness. This construction ensures that performance differences across versions can be attributed to feature engineering rather than to changes in the underlying data.

4.4 Default-Threshold Results Without SMOTE

Table 4.2 reports the best model per version at the default decision threshold without SMOTE, evaluated on the untouched test split. Fraud-class F1 rises from approximately 0.78 for v1 and v2 to between 0.84 and 0.85 for v3 to v5, indicating that richer features improve minority-class detection. Tree-based and boosting models are selected throughout; Logistic Regression is never the best model.

Table 4.2: Best Default-Threshold Results Without SMOTE

Version	Best Model	Precision	Recall	Fraud F1	PR-AUC
v1	XGBoost	0.8062	0.7502	0.7772	0.8268
v2	XGBoost	0.8094	0.7500	0.7786	0.8274
v3	CatBoost	0.9219	0.7739	0.8414	0.8558
v4	Decision Tree	0.9537	0.7678	0.8507	0.8337
v5	Random Forest	0.9418	0.7713	0.8480	0.8561

4.5 Default-Threshold Results With SMOTE

Table 4.3 reports the corresponding best models with SMOTE applied within cross-validation folds. SMOTE raises fraud-class F1 in every version relative to the non-SMOTE setting at the default threshold, principally by increasing precision while modestly reducing recall. The strongest versions, v3 to v5, attain fraud-class F1 of approximately 0.862, and their results are practically indistinguishable from one another.

Table 4.3: Best Default-Threshold Results With SMOTE

Version	Best Model	Precision	Recall	Fraud F1	PR-AUC
v1	Decision Tree	0.9760	0.7040	0.8180	0.8237
v2	Random Forest	0.9527	0.7153	0.8171	0.8265
v3	Random Forest	0.9985	0.7581	0.8618	0.8552
v4	CatBoost	0.9998	0.7570	0.8616	0.8564
v5	CatBoost	0.9999	0.7571	0.8617	0.8558

4.6 Effect of Progressive Feature Engineering

The progression from v1 and v2 to v3, v4 and v5 produces a clear improvement in fraud-class performance. The largest gain accompanies the introduction of the merchant and card risk indicators in v3 and the amount-context features in v4 and v5. Beyond v3, however, the gains are marginal: v3, v4 and v5 are practically tied, so the evidence does not support a claim that v5 is conclusively superior to v3. Figures 4.2 to 4.5 compare the non-SMOTE and SMOTE configurations across versions on fraud-class F1, precision, recall and PR-AUC respectively.



Fig. 4.2: Fraud-class F1 comparison between No-SMOTE and SMOTE configurations.

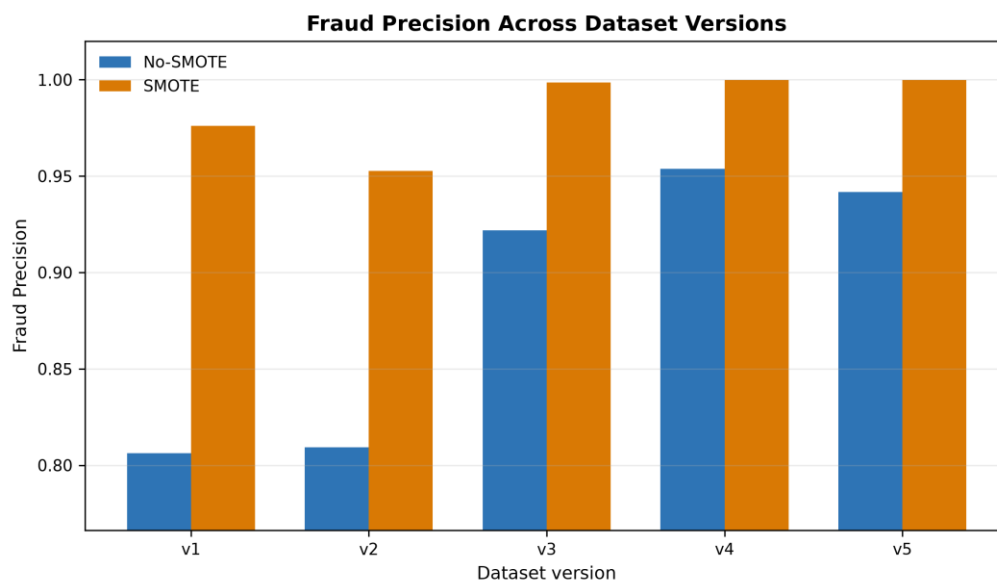


Fig. 4.3: Fraud-class precision comparison between No-SMOTE and SMOTE configurations.



Fig. 4.4: Fraud-class recall comparison between No-SMOTE and SMOTE configurations.

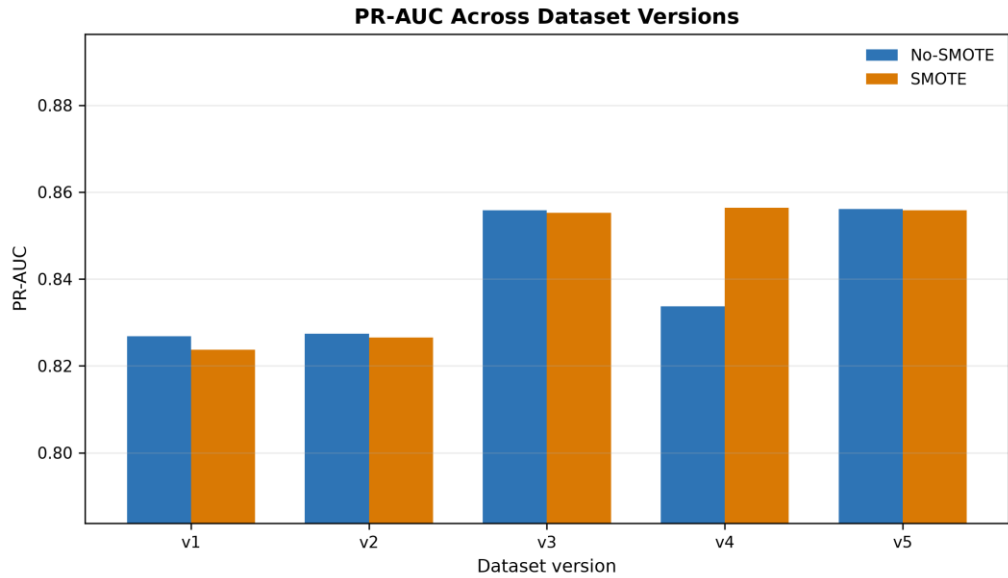


Fig. 4.5: PR-AUC comparison between No-SMOTE and SMOTE configurations.

4.7 Effect of SMOTE

At the default threshold, SMOTE improves fraud-class F1 in all five versions, as shown in Figure 4.2. The mechanism is evident in Figures 4.3 and 4.4: SMOTE raises precision substantially, often to near unity in the strongest versions, while reducing recall modestly. PR-AUC, a threshold-independent measure, is largely unchanged between the two configurations (Figure 4.5), which indicates that SMOTE chiefly shifts the default operating point rather than improving the underlying ranking of transactions. This observation anticipates the threshold-tuning results and cautions

against concluding that SMOTE universally improves fraud detection or universally improves recall.

4.8 Threshold-Tuned Results

When the decision threshold is selected on validation data to maximise fraud-class F1 and then applied to the test data, the apparent advantage of SMOTE largely disappears. Table 4.4 compares the best tuned operating points for the non-SMOTE and SMOTE configurations across versions. The tuned fraud-class F1 values differ by less than 0.001 between the two configurations in every version, confirming that threshold selection, rather than resampling, accounts for most of the default-threshold difference. Figure 4.6 shows the threshold-optimisation curve for the best overall model.

Table 4.4: Threshold-Tuned Comparison

Version	No-SMOTE Model	Thr.	F1	SMOTE Model	Thr.	F1
v1	Random Forest	0.80	0.8191	CatBoost	0.44	0.8190
v2	XGBoost	0.83	0.8188	XGBoost	0.43	0.8188
v3	XGBoost	0.82	0.8617	XGBoost	0.51	0.8616
v4	Decision Tree	0.76	0.8618	XGBoost	0.47	0.8614
v5	Random Forest	0.80	0.8620	XGBoost	0.45	0.8615

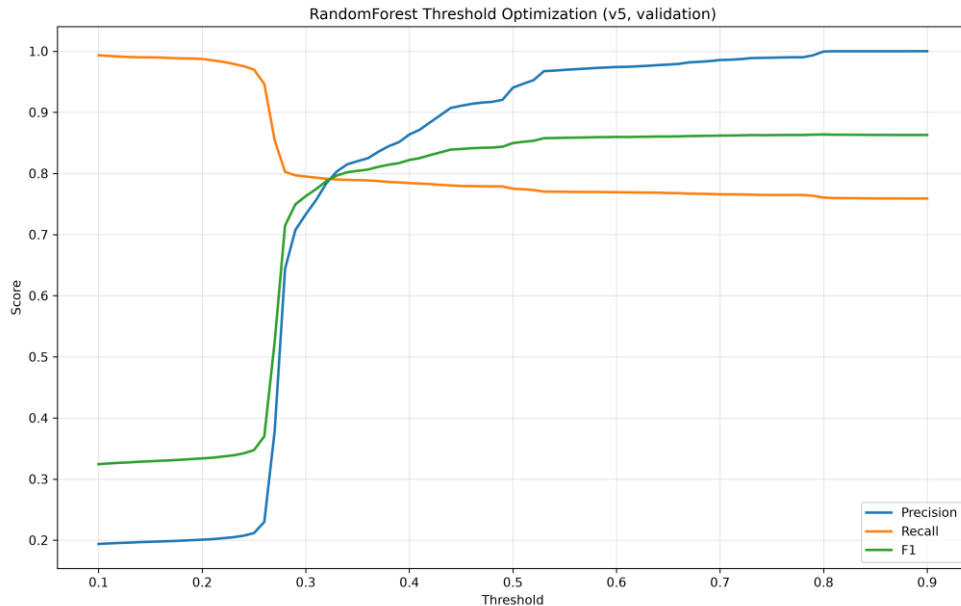


Fig. 4.6: Threshold optimisation curve for the v5 No-SMOTE Random Forest model.

4.9 Best Model Interpretation

The best overall tuned operating point was obtained by the v5 non-SMOTE Random Forest at a threshold of 0.80, with fraud-class precision of 0.9992, recall of 0.7580 and fraud-class F1 of 0.8620 on the untouched test split. At this operating point the model produced 9,475 true positives, 87,489 true negatives, only 8 false positives and 3,025 false negatives, as shown in the confusion matrix in Figure 4.7. The very high precision indicates that almost all flagged transactions are genuinely fraudulent, while the recall of about 0.76 indicates that roughly a quarter of frauds remain undetected at this operating point. It must be emphasised that the v3, v4 and v5 tuned results are practically tied, so the selection of the v5 configuration as best should be read as a marginal, not a decisive, advantage.

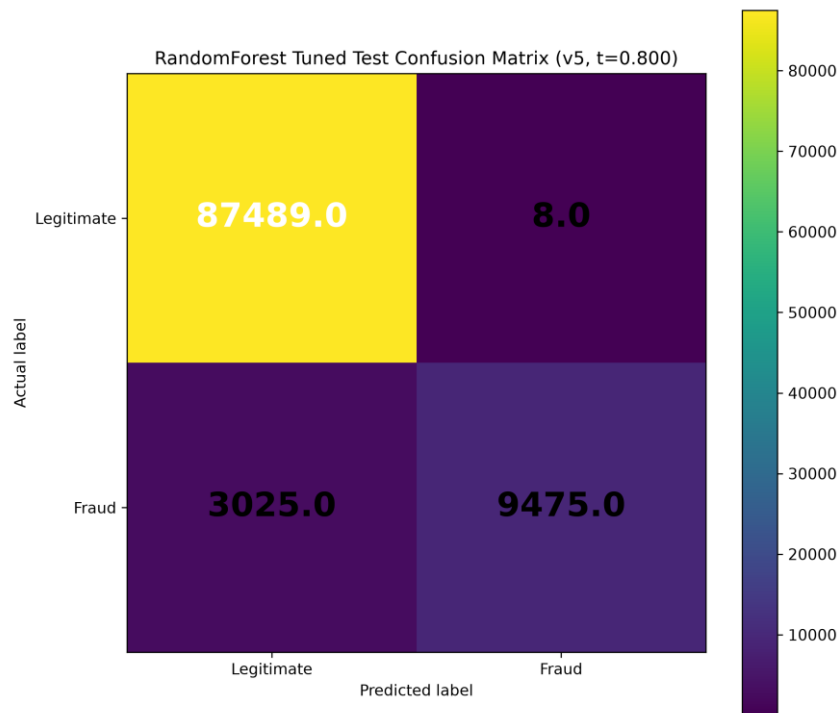


Fig. 4.7: Tuned test confusion matrix for the v5 No-SMOTE Random Forest model at threshold 0.80.

Figure 4.8 shows the impurity-based feature importances for the selected v5 non-SMOTE Random Forest, computed from the saved model artifact. The amount-to-category-median ratio is by far the most influential feature, followed by the merchant identity, the amount percentile and amount-related transforms, with the card risk indicator also contributing. Because impurity-based importance can be biased towards high-cardinality features and does not establish causation, these values are interpreted

as indicative only; a model-agnostic analysis such as SHAP is recommended for future work (Lundberg and Lee, 2017).

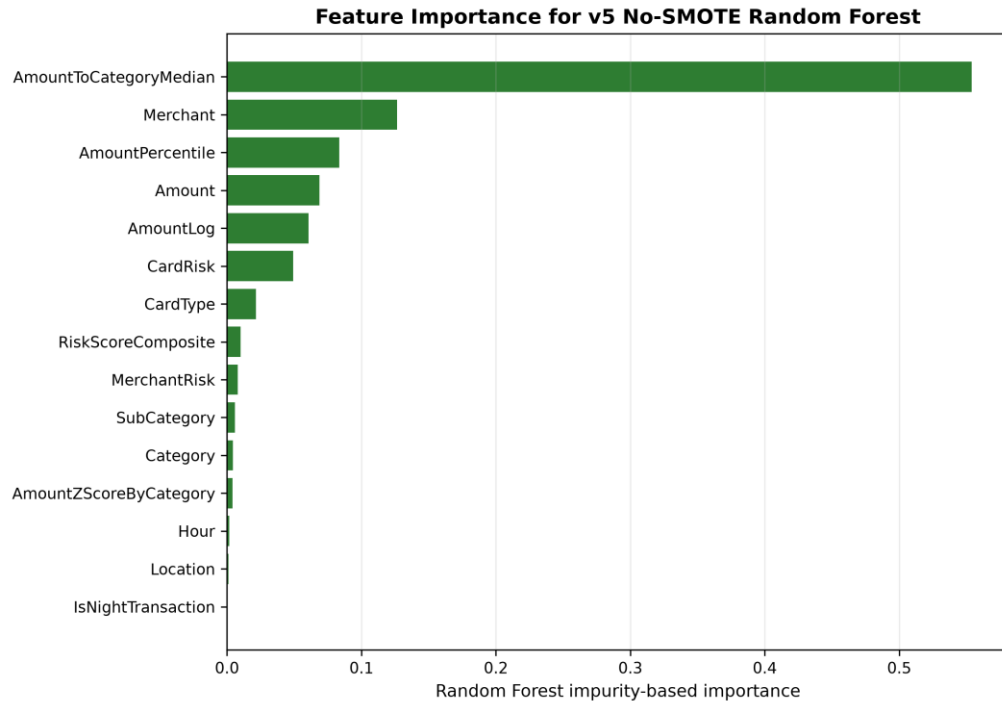


Fig. 4.8: Impurity-based feature importance for the selected v5 No-SMOTE Random Forest model.

4.10 Discussion of Findings in Relation to Research Questions

The findings address each research question directly. The dataset was successfully generated, cleaned and validated, providing a sound and reproducible basis for experimentation (RQ1). Progressive feature engineering improved fraud-class performance, with the clearest gains from v1 and v2 to v3, after which v3 to v5 were practically tied (RQ2). Tree-based and boosting models consistently outperformed Logistic Regression, which was never selected as a best model (RQ3). SMOTE improved fraud-class F1 at the default threshold in all versions, mainly by raising precision at the expense of some recall (RQ4). Validation-based threshold tuning largely removed the practical SMOTE advantage, with tuned F1 values differing by less than 0.001 between configurations (RQ5). The most effective configuration under the study's conditions was the v5 non-SMOTE Random Forest at threshold 0.80, with the important caveat that v3 to v5 are practically tied (RQ6). These outcomes are consistent with the literature emphasising feature engineering, precision-recall evaluation and operating-point selection (Whitrow et al., 2009; Saito and Rehmsmeier, 2015). Table 4.5 consolidates the findings by research question.

Table 4.5: Summary of Findings by Research Question

RQ	Finding	Supporting Evidence
RQ1	Dataset cleaned and validated; no duplicates or missing values	Table 4.1; dataset audit
RQ2	Feature engineering improves fraud-class F1; v3-v5 practically tied	Tables 4.2-4.3; Figs 4.2-4.5
RQ3	Tree/boosting models outperform Logistic Regression	Tables 4.2-4.4 (LR never selected)
RQ4	SMOTE improves default-threshold F1 via higher precision, lower recall	Tables 4.2-4.3; Figs 4.3-4.4
RQ5	Threshold tuning removes most SMOTE advantage (<0.001 difference)	Table 4.4; Fig 4.6
RQ6	Best tuned config v5 No-SMOTE Random Forest (F1 ~ 0.862), v3-v5 tied	Table 4.4; Figs 4.7-4.8

4.11 Summary of Chapter

This chapter showed that the dataset is clean and balanced in design across feature versions, that progressive feature engineering improves fraud-class performance with diminishing returns beyond v3, that SMOTE improves default-threshold F1 chiefly by raising precision, and that validation-based threshold tuning equalises the SMOTE and non-SMOTE configurations. The best tuned model attained a fraud-class F1 of approximately 0.862, while the strongest versions remained practically tied. These results are interpreted cautiously and within the limitations of a synthetic, internally validated study.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary of the Study

This study designed, implemented and evaluated a reproducible, imbalance-aware machine learning framework for credit card fraud detection, with particular attention to the effects of progressive feature engineering and of class-imbalance handling. Using a researcher-generated synthetic dataset of 499,985 transactions with a 12.5004 per cent fraud ratio, five progressively enriched feature versions were constructed from a single clean population. Five classifiers, Logistic Regression, Decision Tree, Random Forest, XGBoost and CatBoost, were trained under a stratified 60/20/20 split with five-fold cross-validated tuning on PR-AUC, validation-based model and threshold selection, and a single untouched test evaluation. Each configuration was assessed without and with SMOTE.

5.2 Summary of Findings

The study found that progressive feature engineering improved fraud-class performance, with the clearest gains achieved by version v3 and only marginal further gains for v4 and v5, which were practically tied with v3. Tree-based and boosting models consistently outperformed the linear baseline, which was never selected as a best model. SMOTE improved fraud-class F1 at the default threshold in every version, chiefly by increasing precision while modestly reducing recall, but threshold-independent PR-AUC was largely unchanged. After validation-based threshold tuning, the SMOTE and non-SMOTE configurations were practically equal, differing by less than 0.001 in tuned fraud-class F1. The best tuned operating point, achieved by the v5 non-SMOTE Random Forest at a threshold of 0.80, attained a fraud-class F1 of approximately 0.862, with precision of about 0.999 and recall of about 0.758.

5.3 Conclusion

It is concluded that, on this dataset, fraud-class performance is governed principally by feature engineering and by the selection of the operating threshold, rather than by over-sampling alone. SMOTE is beneficial at the default threshold but its advantage is largely an artefact of operating-point placement that disappears once thresholds are

tuned. The strongest feature versions are practically indistinguishable, so claims of a single best version should be made cautiously. These conclusions hold within the bounds of a synthetic, internally validated study and should not be generalised to real financial institutions without external validation.

5.4 Contributions to Knowledge

The study makes the following contributions. It demonstrates a transparent and reproducible, leakage-aware pipeline for imbalanced tabular fraud detection, in which distribution-dependent features are recomputed within cross-validation folds and final results are reported once on an untouched test split. It provides controlled evidence on the effect of progressive feature engineering by holding the data population constant. It offers a clear, evidence-based account of how the apparent benefit of SMOTE interacts with threshold selection, contributing to a more cautious and accurate interpretation of resampling in fraud detection.

5.5 Recommendations

On the basis of the findings, the following recommendations are made. Researchers and practitioners should evaluate imbalance strategies in conjunction with threshold tuning, rather than at a single default threshold, and should report fraud-class metrics and PR-AUC rather than accuracy. Feature engineering should be prioritised as a primary lever for improving minority-class detection. Negligible differences between strong configurations should not be over-interpreted. Any move towards real-world use should be preceded by external validation, a leakage audit of proxy risk indicators and an appropriate fairness and regulatory review.

5.6 Suggestions for Future Work

Several extensions would strengthen and broaden this work:

1. external validation on independently sourced, real-world transaction data;
2. repeated-seed experiments to quantify robustness across random initialisations;
3. computation of confidence intervals and formal significance testing for the reported metrics;
4. a dedicated leakage audit of the MerchantRisk and CardRisk proxy indicators;

5. model-agnostic explainability analysis, such as SHAP, to complement impurity-based importance (Lundberg and Lee, 2017);
6. real-time and streaming evaluation under concept drift;
7. cost-sensitive thresholding that incorporates the differing costs of false positives and false negatives; and
8. exploration of deep learning and graph-based fraud models.

REFERENCES

- Abdallah, A., Maarof, M. A. and Zainal, A. 2016. Fraud detection system: a survey. *Journal of Network and Computer Applications* 68: 90-113.
- Bahnsen, A. C., Aouada, D., Stojanovic, A. and Ottersten, B. 2016. Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications* 51: 134-142.
- Bhattacharyya, S., Jha, S., Tharakunnel, K. and Westland, J. C. 2011. Data mining for credit card fraud: a comparative study. *Decision Support Systems* 50.3: 602-613.
- Branco, P., Torgo, L. and Ribeiro, R. P. 2016. A survey of predictive modeling on imbalanced domains. *ACM Computing Surveys* 49.2: 1-50.
- Breiman, L. 2001. Random forests. *Machine Learning* 45.1: 5-32.
- Carcillo, F., Le Borgne, Y.-A., Caelen, O., Kessaci, Y., Oble, F. and Bontempi, G. 2021. Combining unsupervised and supervised learning in credit card fraud detection. *Information Sciences* 557: 317-331.
- Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. 2002. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16: 321-357.
- Chen, T. and Guestrin, C. 2016. XGBoost: a scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM. 785-794.
- Cox, D. R. 1958. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B* 20.2: 215-242.
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C. and Bontempi, G. 2018. Credit card fraud detection: a realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems* 29.8: 3784-3797.
- Dal Pozzolo, A., Caelen, O., Johnson, R. A. and Bontempi, G. 2015. Calibrating probability with undersampling for unbalanced classification. 2015 *IEEE Symposium Series on Computational Intelligence*. IEEE. 159-166.

- Davis, J. and Goadrich, M. 2006. The relationship between precision-recall and ROC curves. *Proceedings of the 23rd International Conference on Machine Learning*. New York: ACM. 233-240.
- Fernandez, A., Garcia, S., Galar, M., Prati, R. C., Krawczyk, B. and Herrera, F. 2018. *Learning from imbalanced data sets*. Cham: Springer.
- Friedman, J. H. 2001. Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29.5: 1189-1232.
- Hand, D. J. 2009. Measuring classifier performance: a coherent alternative to the area under the ROC curve. *Machine Learning* 77.1: 103-123.
- Hastie, T., Tibshirani, R. and Friedman, J. 2009. *The elements of statistical learning: data mining, inference, and prediction*. 2nd ed. New York: Springer.
- He, H. and Garcia, E. A. 2009. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering* 21.9: 1263-1284.
- James, G., Witten, D., Hastie, T. and Tibshirani, R. 2013. *An introduction to statistical learning with applications in R*. New York: Springer.
- Jurgovsky, J., Granitzer, M., Ziegler, K., Calabretto, S., Portier, P.-E., He-Guelton, L. and Caelen, O. 2018. Sequence classification for credit-card fraud detection. *Expert Systems with Applications* 100: 234-245.
- Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. 2012. Leakage in data mining: formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6.4: 1-21.
- Krawczyk, B. 2016. Learning from imbalanced data: open challenges and future directions. *Progress in Artificial Intelligence* 5.4: 221-232.
- Lemaitre, G., Nogueira, F. and Aridas, C. K. 2017. Imbalanced-learn: a Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research* 18.17: 1-5.
- Lundberg, S. M. and Lee, S.-I. 2017. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* 30: 4765-4774.

- Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y. and Sun, X. 2011. The application of data mining techniques in financial fraud detection: a classification framework and an academic review of literature. *Decision Support Systems* 50.3: 559-569.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12: 2825-2830.
- Phua, C., Lee, V., Smith, K. and Gayler, R. 2010. A comprehensive survey of data mining-based fraud detection research. *arXiv preprint arXiv:1009.6119*.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. and Gulin, A. 2018. CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems* 31: 6638-6648.
- Provost, F. and Fawcett, T. 2001. Robust classification for imprecise environments. *Machine Learning* 42.3: 203-231.
- Quinlan, J. R. 1986. Induction of decision trees. *Machine Learning* 1.1: 81-106.
- Saito, T. and Rehmsmeier, M. 2015. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE* 10.3: e0118432.
- West, J. and Bhattacharya, M. 2016. Intelligent financial fraud detection: a comprehensive review. *Computers and Security* 57: 47-66.
- Whitrow, C., Hand, D. J., Juszczak, P., Weston, D. and Adams, N. M. 2009. Transaction aggregation as a strategy for credit card fraud detection. *Data Mining and Knowledge Discovery* 18.1: 30-55.

APPENDICES

Appendix A: Dataset Features / Data Dictionary

Table A.1 describes the principal fields available across the dataset versions. Calendar fields and identifiers listed as excluded were not used as model inputs.

Table A.1: Data Dictionary

Feature	Type	Description	Introduced
TransactionID	Identifier	Deterministic hashed transaction identifier (excluded from models)	v1
Amount	Numeric	Transaction amount	v1
Merchant	Categorical	Merchant associated with the transaction	v1
Category	Categorical	Transaction category	v1
CardType	Categorical	Type of payment card	v1
Location	Categorical	Transaction location	v1
Time	Temporal	Transaction time (excluded from models)	v1
FraudFlag	Target	1 = fraud, 0 = legitimate	v1
HighAmountFlag	Binary	Indicator of unusually high amount	v2
IsNightTransaction	Binary	Indicator of a night-time transaction	v2
SubCategory	Categorical	Finer transaction subcategory	v2
MerchantRisk	Numeric	Proxy merchant risk indicator (leakage-cautious)	v3
CardRisk	Numeric	Proxy card risk indicator (leakage-cautious)	v3
Hour	Numeric	Hour of transaction	v4
AmountLog	Numeric	Logarithmic transform of amount	v4
AmountToCategoryMedian	Numeric	Amount relative to category median	v4
AmountZScoreByCategory	Numeric	Standardised amount within category	v4
AmountPercentile	Numeric	Percentile rank of amount	v4
NightHighAmountFlag	Binary	Night and high-amount interaction	v5
RiskScoreComposite	Numeric	Composite weighted risk score	v5

Appendix B: Experimental Configuration

All experiments used a fixed random state of 42 for reproducibility. Hyperparameters were tuned by randomised search with five-fold stratified cross-validation on the training split, optimising average precision (PR-AUC). Search spaces covered the principal complexity and regularisation parameters of each model family, including the regularisation strength for Logistic Regression; tree depth, splitting and pruning parameters for the Decision Tree; the number of trees, depth, feature-sampling and bootstrap settings for the Random Forest; the number of estimators, learning rate,

depth, subsampling, column sampling and regularisation for XGBoost; and the number of iterations, learning rate, depth and leaf regularisation for CatBoost. Model-family selection followed a documented fraud-class-F1-first rule with practical tolerances on PR-AUC and recall, with accuracy never used for selection. The full experimental configuration is summarised in Table 3.3.

Appendix C: Additional Metrics

Table C.1 reports supplementary test-split metrics for the best non-SMOTE model in each version, namely overall accuracy and ROC-AUC, complementing the fraud-class metrics in Chapter Four. Accuracy is reported for completeness only and was not used for model selection.

Table C.1: Supplementary Test Metrics (Best No-SMOTE Model per Version)

Version	Best Model	Accuracy	ROC-AUC	PR-AUC
v1	XGBoost	0.9462	0.9112	0.8268
v2	XGBoost	0.9467	0.9120	0.8274
v3	CatBoost	0.9635	0.9312	0.8558
v4	Decision Tree	0.9663	0.9295	0.8337
v5	Random Forest	0.9654	0.9314	0.8561

Appendix D: Reproducibility Notes

The pipeline was implemented in Python using established open-source libraries, principally scikit-learn for modelling and evaluation (Pedregosa, Varoquaux, Gramfort, Michel, Thirion, Grisel, Blondel, Prettenhofer, Weiss, Dubourg, Vanderplas, Passos, Cournapeau, Brucher, Perrot and Duchesnay, 2011), imbalanced-learn for SMOTE (Lemaitre et al., 2017), and the XGBoost and CatBoost gradient-boosting libraries (Chen and Guestrin, 2016; Prokhorenkova et al., 2018), together with pandas, NumPy and Matplotlib for data handling and visualisation. Reproducibility was supported by a fixed random seed (42), deterministic hashed transaction identifiers, stratified splitting, leakage-safe in-fold feature recomputation, and the persistence of metrics, predictions and trained model artifacts for each version and configuration. Verification confirmed that the training, validation and test partitions shared no transaction identifiers.

Appendix E: Selected Code / Workflow Summary

The experimental workflow was organised into documented stages. A dataset-generation stage rebuilt the five feature versions from a single clean master population with deterministic identifiers. A core modelling module defined the leakage-safe feature engineering, the model-specific preprocessing, the SMOTE-enabled pipelines, the cross-validated randomised search and the fraud-class-first model-selection rule. An experiment-runner stage executed the standard and SMOTE configurations across all versions, producing per-model metrics, prediction files, confusion matrices and ROC and precision-recall curves. A post-training stage performed validation-based threshold optimisation and compiled the comparison outputs. The figures and tables presented in this dissertation were derived directly from these saved outputs.